



Ingegneria del software

Capitolo 1: Introduzione all'Ingegneria del Software

L'ingegneria del software nasce dalla necessità di gestire la complessità intrinseca nella produzione di sistemi di grandi dimensioni, spesso caratterizzati da un ciclo di vita non terminante. A differenza della semplice programmazione, l'obiettivo primario non è la mera scrittura del codice, bensì la definizione di un'architettura solida e di specifiche di sistema rigorose.

1.1 Attori e Processi nel Ciclo di Vita

La produzione del software coinvolge tre figure chiave che interagiscono in un ciclo di feedback continuo:

Customer (Cliente): Definisce le necessità del sistema e valuta tempi e costi.

Software Engineer: Traduce i requisiti in specifiche tecniche e progetta l'architettura.

Developer (Sviluppatore): Implementa fisicamente il sistema e ne verifica i risultati.

Modellizzazione con UML

Per descrivere la dinamica e il comportamento del software, si utilizza il linguaggio **UML** (Unified Modeling Language). I diagrammi principali includono:

- **Context Diagram:** Rappresenta l'insieme dei servizi offerti dal sistema e le sue interazioni esterne.
- **Activity Diagram:** Descrive l'evoluzione sequenziale o parallela di un task specifico (es. l'inserimento di un paziente in una clinica).
- **Sequence & Class Diagrams:** Utilizzati per modellare rispettivamente le interazioni temporali tra oggetti e la struttura statica del sistema.

1.2 Modellazione Predittiva: Le Catene di Markov

Nella progettazione moderna, si adotta il concetto di **Digital Twin**: una copia digitale del software utilizzata per simulare scenari operativi e condurre test automatici prima della produzione. Per sistemi discreti a stati (finiti o infiniti), lo strumento matematico d'elezione è la **Catena di Markov a Tempo Discreto (DTMC)**.

Definizione Formale di DTMC

Una Catena di Markov Discreta è definita come una tupla (U, X, Y, p, g) dove

- U : Insieme degli ingressi (input).
- X : Insieme non vuoto degli stati del sistema.
- Y : Insieme non vuoto delle uscite (output).
- p : Funzione di probabilità di transizione, definita come:
 $p: X \times X \times U \rightarrow [0, 1]$
dove $p(x' | x, u)$ indica la probabilità di passare allo stato x' partendo da x con ingresso u .
Per ogni coppia (x, u) , deve valere la condizione di normalizzazione:
$$\sum_{x' \in X} p(x' | x, u) = 1$$
- g : Funzione di output $g: X \rightarrow Y$.

Analisi Predittiva (Esempi Pratici)

Le DTMC permettono di calcolare la probabilità di completamento di un progetto entro certi vincoli temporali o di costo:

1. **Tempo di Completamento**: Dato un processo suddiviso in fasi (es. Requisiti, Sviluppo, Collaudo), la probabilità di terminare in N mesi è la somma delle probabilità di tutti i cammini i cui tempi totalizzano N .

• Esempio: $p(\text{finire in 10 mesi}) = 0.1008$.
2. **Costi di Design**: È possibile determinare la probabilità minima di successo di una fase (parametro p) affinché il progetto rispetti un budget o una scadenza.

• Esempio: Per finire in 8 mesi con probabilità ≥ 0.5 , si ricava $p \geq 0.8928$ tramite l'equazione:
$$1 \cdot p \cdot 0.8 \cdot 0.7 \geq 0.5$$

1.3 Interconnessione di Sistemi e Reti di Markov

I sistemi complessi sono spesso composti da più sottosistemi interconnessi tramite i rispettivi ingressi e uscite, formando una **Rete di Catene di Markov**.

Siano K e P due catene dove l'ingresso di una è l'uscita dell'altra. Il sistema globale equivalente M è definito come:

$$M = (V, X_1 \times X_2, Z, p, g)$$

La funzione di probabilità congiunta del sistema è data dal prodotto delle probabilità delle singole catene, considerando la mutua dipendenza degli ingressi dalle uscite correnti:

$$p((x_1', x_2') | (x_1, x_2), v) = p_1(x_1' | x_1, (g_2 w(x_2), v)) \cdot p_2(x_2' | x_2, g_1(x_1))$$

1.4 Implementazione Software (C++)

L'implementazione di una DTMC in C++ richiede una struttura a classi per incapsulare lo stato e le funzioni di transizione.

Struttura della Classe `DTMC`

La classe gestisce vettori dinamici per stati, ingressi e uscite:

- **Membri Privati:** Un puntatore a un generatore di numeri casuali (`RandGen *p`) e il core logico `dtmc_body`.
- **Membri Pubblici:** Vettori `state_type *x`, `input_type *u`, `output_type *y` e le dimensioni `STATE_SIZE`, `INPUT_SIZE`, `OUTPUT_SIZE`.

Ciclo Operativo del Simulatore

Il simulatore esegue la catena per un orizzonte temporale T :

1. `init()`: Inizializzazione dello stato (tipicamente a zero).
 2. `output()`: Calcolo dell'uscita corrente basata sullo stato.
 3. `next()`: Calcolo dello stato successivo campionando da una distribuzione casuale.
- *Nota:* Se lo stato è un vettore di numeri reali continui, la DTMC è a stati infiniti.

```
// Esempio di retroazione nel simulatore
for (t = 0; t < T; t++) {
    mcl.output();
    mcl.u[1] = mcl.y[1] + 1; // Sistema a retroazione:  $u(t) = y(t-1) + 1$ 
    mcl.next();
}
```

Capitolo 2: Planning del Progetto

Il planning consiste nella scomposizione del carico di lavoro tra i membri del team, con l'obiettivo di prevedere criticità e definire strategie di mitigazione. È un processo fondamentale per comunicare la struttura del progetto sia internamente che al cliente.

2.1 Fasi e Scopo della Pianificazione

La pianificazione non è un evento isolato, ma avviene in tre momenti chiave:

- **Proposta:** Durante la contrattazione commerciale con il cliente.
- **Avvio:** Quando si allocano le risorse e si assegnano i task.
- **Sviluppo:** Periodicamente, per monitorare i progressi e aggiornare i piani in base all'andamento reale.

Lo scopo primario è la definizione del budget, stimando i costi relativi a personale, hardware e software.

2.2 Sviluppo Plan-Driven

L'approccio **Plan-Driven** (o guidato dal piano) prevede che i processi di sviluppo siano programmati dettagliatamente fin dall'inizio. Si basa su tecniche ingegneristiche classiche per gestire grandi progetti.

Sezioni di un Project Plan

Un piano strutturato deve includere:

1. **Analisi dei rischi:** Identificazione dei potenziali ostacoli.
2. **WBS (Work Breakdown Structure):** Struttura di scomposizione del lavoro.
3. **Project Schedule:** Il calendario delle attività.

A supporto del piano principale, vengono redatti piani specifici come il *Configuration management plan*, il *Deployment plan* e il *Maintenance plan*.

Il Processo Iterativo del Planning

Nonostante la rigidità apparente, il planning è iterativo. Se insorgono problemi gravi, il manager deve intervenire con azioni di mitigazione del rischio o procedere alla **riplanificazione totale**, rinegoziando eventualmente i termini con il cliente.

2.3 Scheduling del Progetto

Lo scheduling è il processo di decisione sull'organizzazione delle attività e sull'ordine della loro esecuzione.

Parametri delle Attività

Ogni "Project Activity" è caratterizzata da:

- **Durata:** Giorni o mesi previsti sul calendario.
- **Sforzo (Effort):** Misurato in persone/giorno.
- **Deadline:** Vincolo temporale di completamento.
- **Milestone:** Tappa fondamentale per la valutazione del progresso.

Strumenti Grafici

Per rappresentare lo scheduling si utilizzano due diagrammi principali:

- **Activity Bar Chart (Diagramma di Gantt):** Mostra le attività sull'asse temporale rispettando le dipendenze.
 - **Staff Allocation Chart:** Illustra come il personale è distribuito tra i vari task nelle diverse settimane.
-

2.4 Sviluppo Agile e Planning

A differenza del plan-driven, lo **Sviluppo Agile** è iterativo: i requisiti e il piano vengono modellati dinamicamente durante il lavoro. Le decisioni non sono predeterminate, ma dipendono dal feedback costante del cliente.

Livelli di Pianificazione Agile

Lo sviluppo agile si articola in due stadi:

1. **Release Planning:** Pianificazione a lungo termine (mesi) delle funzionalità da implementare.

2. **Iteration Planning:** Focus a breve termine (2-4 settimane) su incrementi specifici, spesso chiamati *sprint*.

Story-Based Planning

Questa metodologia suddivide le funzionalità in **User Stories**:

- **Stima dello sforzo:** Il team assegna alle storie dei valori numerici chiamati *effort points*.
- **Task Planning:** Gli sviluppatori suddividono le storie in compiti atomici da 4-16 ore, offrendosi volontari per la loro implementazione.
- **Velocità del Team:** Quantità di lavoro (effort points) completata in un'iterazione, usata per stimare i tempi totali.

Vantaggi e Limiti

- **Vantaggio:** Viene sempre consegnato un incremento di software funzionante alla fine di ogni iterazione.
 - **Svantaggio:** Richiede un forte coinvolgimento e disponibilità del cliente, che non sempre è possibile in organizzazioni complesse o con team distribuiti.
-

Capitolo 3: Processi Software

Il processo software è definito come un insieme strutturato di attività necessarie per sviluppare un sistema. Sebbene esistano approcci diversi, tutti condividono fasi fondamentali: specifica, progettazione/implementazione, validazione ed evoluzione.

3.1 Classificazione dei Modelli

I processi possono essere classificati in due macro-categorie:

- **Plan-driven:** Tutte le attività sono pianificate in anticipo e il progresso è misurato rispetto a un piano iniziale.
- **Agili:** La pianificazione è incrementale e flessibile, adattandosi ai cambiamenti dei requisiti attraverso iterazioni brevi (sprint).

3.2 Modelli di Sviluppo Principali

Vengono analizzati i tre modelli cardine dell'ingegneria del software:

1. Waterfall Model (Modello a Cascata):

- Approccio plan-driven che separa rigidamente le fasi.
- Le fasi includono: Analisi requisiti, Progettazione, Implementazione, Integrazione e Manutenzione.
- **Svantaggio:** Scarsa flessibilità nel gestire cambiamenti a progetto avviato.

2. Incremental Development:

- Punta alla riduzione dei costi per accomodare nuovi requisiti in itinere.
- Permette feedback rapidi e consegne anticipate di software utile.
- **Svantaggio:** Può portare al degrado della struttura del codice se non mantenuto correttamente.

3. Integration and Configuration (Orientato al Riuso):

- Sviluppo basato sulla combinazione di componenti esistenti o sistemi COTS (*Commercial-off-the-shelf*).
 - **Vantaggi:** Costi e rischi ridotti grazie al minor volume di codice scritto da zero.
-

Capitolo 4: Sviluppo Agile

4.1 La Filosofia e il Contesto

Nel mercato attuale, i requisiti di un software cambiano troppo rapidamente perché lo sviluppo "tradizionale" (plan-driven) possa stare al passo. Lo sviluppo agile nasce alla fine degli anni '90 con l'obiettivo di ridurre i tempi di consegna, fornendo software funzionante in tempi brevissimi.

Concetti Chiave

- **Interleaving (Intreccio delle fasi):** Specifica, progettazione e implementazione non sono sequenziali, ma si sovrappongono e si influenzano a vicenda.
 - **Sviluppo Incrementale:** Il sistema cresce per versioni successive, migliorando a ogni passo grazie al feedback degli stakeholder.
 - **Consegne Frequenti:** Il rilascio costante di nuove versioni permette di correggere subito la rotta.
 - **Documentazione Minima:** Si dà priorità al codice che funziona. La documentazione si scrive solo se strettamente necessaria.
-

4.2 Agile vs. Plan-Driven

Il documento chiarisce la differenza strutturale tra i due approcci:

- **Plan-driven:** Segue un approccio sequenziale e una pianificazione anticipata rigida. Ogni fase ha risultati previsti definiti a tavolino.
 - **Agile:** Le iterazioni sono costanti. Se cambiano i requisiti (fatto considerato inevitabile), il processo torna immediatamente alla fase di ingegneria dei requisiti per riadattare la progettazione e l'implementazione.
-

4.3 L'Agile Manifesto e i suoi Principi

I metodi agili si basano su quattro valori fondamentali che privilegiano:

1. **Individui e interazioni** rispetto a processi e strumenti.
2. **Software funzionante** rispetto a documentazione esaustiva.
3. **Collaborazione con il cliente** rispetto alla negoziazione contrattuale.
4. **Risposta al cambiamento** rispetto al seguire un piano fisso.

Principi Operativi

- **Coinvolgimento del cliente:** Il cliente non è solo un committente, ma fornisce requisiti e valuta ogni iterazione.
- **Priorità alle persone:** Si sfruttano le competenze del team, lasciandoli liberi di organizzare il proprio lavoro senza processi prescrittivi.
- **Mantenere la semplicità:** Si lavora attivamente per eliminare la complessità inutile sia nel software che nel processo.

4.4 Tecniche: Extreme Programming (XP)

La tecnica più influente analizzata è l'**Extreme Programming**, che porta all'estremo i principi dello sviluppo iterativo.

Caratteristiche di XP

- **Rilasci frequenti:** Nuove versioni anche più volte al giorno.
- **Test continui:** Ogni build deve superare test automatizzati per essere accettata.
- **Design semplice:** Si progetta solo quanto serve per i requisiti attuali.

Pratiche Fondamentali

1. **Sviluppo guidato dai test (Test-Driven Development):** Si scrivono i test *prima* del codice.
2. **Refactoring:** Gli sviluppatori ristrutturano costantemente il codice per migliorarlo e mantenerlo semplice.
3. **Programmazione a coppie (Pair Programming):** Due sviluppatori lavorano insieme sulla stessa postazione per garantire qualità e supporto reciproco.
4. **Proprietà collettiva:** Tutti possono modificare qualsiasi parte del codice, evitando "isole di competenza".
5. **Cliente in loco:** Un rappresentante degli utenti è sempre disponibile per il team.

4.5 User Stories e Task

Il capitolo si chiude spiegando come vengono gestiti i requisiti in XP:

- **User Stories:** Sono scenari scritti dal punto di vista dell'utente (es. la gestione della "Prescrizione della Medicazione").
- **Scomposizione in Task:** Ogni storia viene frammentata in compiti tecnici più piccoli. Ad esempio, dalla storia della prescrizione derivano:
 - **Task 1:** Cambio del dosaggio.
 - **Task 2:** Selezione del formulario.
 - **Task 3:** Controllo del dosaggio (verifica di sicurezza per limiti minimi e massimi).

Parte pratica (c++)

Preambolo. Model Based Software Design

La necessità dei modelli

Le scelte di progettazione nei progetti software complessi non possono basarsi solo sull'esperienza o sul ragionamento intuitivo. È necessario costruire un **modello del progetto** per confrontare diverse soluzioni tecniche prima di impegnarsi effettivamente in una scelta di design.

Il dilemma "Amazon Prime Video"

Viene presentato un caso di studio reale per illustrare l'utilità della modellazione:

- **Contesto:** Amazon Prime Video doveva decidere tra un'architettura a **microservizi** (**distribuita**) o una **monolitica** per il monitoraggio audio/video.

- **Mito da sfatare:** Comunemente si pensa che i monoliti siano più costosi e meno scalabili.
- **Risultato:** Passando a un'architettura monolitica, il team di Prime Video ha ridotto i costi dell'infrastruttura del **90%**.
- **Conclusione:** Non esiste una risposta definitiva valida per tutti; lo strumento per decidere è la simulazione del sistema modellato come un insieme di **processi decisionali di Markov (MDP)**.

1. Catene di Markov (Markov Chains)

- **Definizione:** Una catena di Markov M è una coppia (S,p) , dove S è l'insieme degli stati e p è la probabilità di transizione.
- **Proprietà di Markov (Memorylessness):** Il futuro del processo dipende esclusivamente dallo stato presente e non dalla storia delle transizioni passate.
- **Vincolo di Probabilità:** Per ogni stato, la somma delle probabilità di transizione verso tutti gli stati possibili deve essere uguale a 1:

$$\forall s \in S, \sum_{s' \in S} p(s'|s) = 1$$
- **DTMC (Discrete-Time Markov Chain):** Si definisce tale se i passi temporali e lo spazio degli stati sono numerabili.
- **Rappresentazione:** Possono essere scritte sotto forma di **matrice di transizione**, dove ogni cella indica la probabilità di passare dallo stato di riga allo stato di colonna.

2. Processi Decisionali di Markov (MDP)

- **Differenza dalle Catene:** Un MDP interagisce con un ambiente esterno tramite input.
- **Struttura (Tupla):** È definito da (U,Y,X,p,g) dove:
 - U : Insieme dei valori di input.
 - Y : Insieme dei valori di output.
 - X : Insieme degli stati.
 - p : Probabilità di transizione $X \times X \times U \rightarrow [0,1]$.
 - g : Funzione di output $X \rightarrow Y$.
- **Regola di transizione:** La somma delle probabilità deve essere 1 per ogni specifico valore di input.
- **Reti di MDP:** Più MDP possono essere connessi in una rete, che mantiene a sua volta le proprietà di un MDP.

3. Analisi Numerica: Media Incrementale

- **Problema della Media Tradizionale:** Sommare tutti i valori prima della divisione può causare un **overflow** numerico (Inf), anche se il risultato finale rientrerebbe nei limiti dello standard IEEE-754.
- **Algoritmo Incrementale:** Permette di calcolare la media aggiungendo un valore alla volta senza accumulare somme enormi:

$$x_n = x_{n-1} \cdot n + x_n$$
- **Vantaggio:** I numeri sommati sono più piccoli e gestibili, evitando errori di precisione o overflow.

4. Algoritmo Online di Welford

- **Scopo:** Calcolare la deviazione standard in modo incrementale, risparmiando memoria e tempo di calcolo.
 - **Formulazione:** Si basa sull'aggiornamento della somma dei quadrati delle differenze dalla media ($M2, n$) a ogni nuovo punto dati ricevuto.
 - **Calcolo finale:** Se $n > 0$, la deviazione standard è $M2, n/n$.
-

5. Metodo di Eulero (Forward Euler)

- **Utilizzo:** Approssimazione della soluzione di equazioni differenziali ordinarie quando non è possibile risolverle analiticamente.
- **Equazione di aggiornamento:** Il valore successivo è dato dal valore attuale più il prodotto tra il passo temporale (Δ) e la derivata nel punto attuale:
$$y_n = y_n + \Delta \cdot f(x_n, y_n)$$
- **Precisione:** Un valore di Δ più piccolo produce un'approssimazione migliore della funzione reale.

6. Metodo Monte Carlo

- **Definizione:** Classe di algoritmi computazionali che si basano sul campionamento casuale ripetuto per ottenere risultati numerici.
- **Applicazioni:** Utilizzato principalmente per problemi di ottimizzazione, integrazione numerica e generazione di campioni da una distribuzione di probabilità.
- **Funzionamento:** Si basa sulla legge dei grandi numeri; eseguendo un gran numero di esperimenti indipendenti e identicamente distribuiti (i.i.d.), la media dei risultati tende a corrispondere al valore atteso della distribuzione.
- **Parallelismo:** È considerato un problema "imbarazzantemente parallelo" (embarrassingly parallel), poiché ogni esperimento è indipendente dagli altri e può essere eseguito facilmente su cluster HPC.

Esempio Pratico: "No budget left"

- **Scenario:** Un software ha un budget rimanente di 550€. Il costo per sviluppare una nuova funzionalità segue una distribuzione uniforme tra 300€ e 1000€.
- **Modellazione:** Il problema viene modellato con una distribuzione di Bernoulli:
 - Valore 1 se il costo è $< 550\text{€}$.
 - Valore 0 se il costo è $\geq 550\text{€}$.
- **Calcolo:** La probabilità p cercata è il rapporto tra il numero di esperimenti con valore 1 e il numero totale di esperimenti.

C++ per la Modellazione: Concetti Base

- **Prerequisiti:** Conoscenza di C e dei principi della programmazione orientata agli oggetti (OOP).
- **Standard Library:** La libreria `<random>` è lo strumento principale per implementare i processi di Markov e gli MDP discussi in precedenza.

Operator Overloading (Sovraccarico degli Operatori)

- **Definizione:** Capacità del C++ di definire comportamenti personalizzati per operatori standard (come `+`, `+=`, `<`, `()`, `[]`) quando applicati a oggetti o strutture.
- **Esempi comuni:**
 - `std::cout << "Hello"` utilizza il sovraccarico di `operator<<` per stampare stringhe invece di eseguire uno shift di bit.
 - L'operatore `()` permette di utilizzare un oggetto come se fosse una funzione (functor), tecnica usata internamente dai motori di generazione casuale.

Gestione della Casualità in C++

- **Random Engines:** Si raccomanda di non usare la funzione `random()` del C, ma i motori C++ (es. `std::default_random_engine`) perché sono veloci, deterministici (dato un seme) e compatibili con le distribuzioni.
- **Random Device:** `std::random_device` è un generatore non deterministico che sfrutta l'entropia hardware (se disponibile).
- **Flusso di lavoro tipico:**
 1. Istanziare un `random_device`.
 2. Usarlo per generare un **seme** (seed).
 3. Inizializzare un motore di numeri pseudo-casuali (`random_engine`) con tale seme.
- **Alias Utile:** Nel testo viene spesso definito `using urng_t = std::default_random_engine;` per brevità (Uniform Random Number Generator type).

Implementazione di MDP e Matrici di Transizione

- **Simulazione di MDP:** In C++, un MDP (come quello dello sviluppo software) può essere simulato utilizzando un array di `std::discrete_distribution`.
- **Matrice di Transizione:** Ogni riga della matrice corrisponde a uno stato e contiene la distribuzione delle probabilità di transizione verso gli stati successivi.
- **Esecuzione:** Il sistema evolve campionando il `next_state` dalla distribuzione associata al `current_state` utilizzando un motore di numeri casuali.

Distribuzione Uniforme Discreta (`std::uniform_int_distribution`)

- **Utilizzo:** Genera numeri interi in un intervallo $[a, b]$ dove ogni valore ha la stessa probabilità.
- **Proprietà Inclusiva:** A differenza di altre librerie, in C++ questa distribuzione è **inclusiva** per entrambi gli estremi; dimenticare di sottrarre 1 dalla dimensione totale (es. `STATES_SIZE - 1`) può causare errori di segmentazione.
- **Applicazioni:** Utile per generare intervalli di attesa tra eventi (es. invio dati ogni 20-30 secondi) o per selezionare uno stato casuale in un MDP.

Distribuzioni Uniforme Continua e Bernoulli

- **Uniforme Continua** (`std::uniform_real_distribution`): Genera numeri reali in un intervallo $[a,b)$.
- **Bernoulli** (`std::bernoulli_distribution`):
 - Restituisce un valore booleano (`true` / `false`) basato su una probabilità p .
 - È ideale per modellare eventi binari, come il fallimento casuale di una richiesta in un protocollo di rete (es. probabilità di errore $p=0.001$).

Distribuzione Normale ed Esponenziale

- **Normale** (`std::normal_distribution`): Richiede come parametri la **media** (μ) e la **deviazione standard** (σ).
- **Esponenziale** (`std::exponential_distribution`):
 - Utilizzata per modellare il tempo intercorrente tra eventi che avvengono a un tasso costante λ .
 - **Parametro Lambda**: Se un sistema riceve 5 richieste al minuto, il tasso al secondo sarà $\lambda=5/60 \approx 0.083$.
 - **Applicazione**: Fondamentale per testare se l'architettura di un server può gestire il carico di richieste generate dai client a intervalli casuali.

Distribuzione di Poisson (`std::poisson_distribution`)

- **Definizione**: Genera il numero di eventi che si verificano in un'unità di tempo fisso, dato un tasso medio di occorrenza.
- **Relazione**: È strettamente correlata alla distribuzione esponenziale: mentre l'esponenziale modella il *tempo tra gli eventi*, Poisson modella il *numero di eventi* in un intervallo.

Distribuzione Discreta e Matrici di Transizione

(`std::discrete_distribution`)

- **Funzionamento**: Questa distribuzione assegna a ogni intero i una probabilità proporzionale a un "peso" fornito in input.
- **Vantaggio**: Non è necessario calcolare manualmente le percentuali; la classe normalizza i pesi in modo che la loro somma sia sempre 1.
- **Mapping**: Si utilizzano spesso gli `enum` per mappare gli indici interi (0, 1, 2...) a nomi significativi (es. Shirt, Hoodie, Pants).
- **Matrici di Transizione**: Per modellare una catena di Markov, si crea un vettore di distribuzioni discrete, dove ogni elemento dell'array rappresenta uno stato e le sue probabilità di spostarsi verso gli altri.
- **Inizializzazione Dinamica**: Se i pesi non sono noti a priori, la distribuzione può essere inizializzata passando i puntatori di inizio e fine (`begin`, `end`) di un contenitore dinamico come un `std::vector`.

Esempio MDP Semplice ("course/1100")

- **Caratteristiche:** Il primo esempio didattico modella un MDP dove ogni stato è un vettore di due numeri reali compresi tra 0 e 1.
- **Transizione:** La probabilità di transizione è uniforme su tutto lo spazio degli stati per ogni input.
- **Implementazione:** Viene utilizzato un `std::vector<real_t>` per mantenere lo stato e una `std::uniform_real_distribution` per aggiornare i componenti a ogni passo temporale.

Reti di MDP - Parte 1 ("course/1200")

- **Definizione:** Sistema composto da due MDP (M1 e M2) che comunicano tra loro.
- **Sincronizzazione:** In questo modello, lo stato futuro di un MDP dipende dallo stato attuale dell'altro.
- **Logica di Connessione:**
 - $X1(t)$ viene calcolato moltiplicando le componenti dello stato di M2 per un valore casuale.
 - $X2(t)$ viene calcolato sommando un valore casuale alle componenti dello stato di M1.
- **Strutturazione del Codice:** Per sistemi piccoli si possono usare variabili dirette, ma per sistemi complessi è preferibile progettare moduli separati per ogni componente e gestire le connessioni esplicitamente.

Note Tecniche per il C++

- **Type Aliases:** L'uso di `using urng_t = std::default_random_engine;` e `typedef double real_t;` è standard negli esempi per mantenere il codice leggibile.
- **Enum vs Enum Class:**
 - `enum`: permette la conversione implicita a intero, utile per mappare stati direttamente agli indici delle matrici.
 - `enum class`: più sicuro (typesafe), ma richiede un cast esplicito (`static_cast`) per ottenere il valore numerico.

Reti di MDP Deterministiche ("course/1300")

- **Caratteristiche:** A differenza degli esempi precedenti, questo modello non coinvolge probabilità.
- **Input e Output:** Gli insiemi di input, output e stati coincidono con lo spazio dei numeri reali $R \times R$.
- **Connessioni Complesse:** L'evoluzione dello stato di un componente è definita da combinazioni lineari delle sue stesse componenti. Ad esempio, una componente dello stato può essere aggiornata come $0.7 \cdot x_0 + 0.7 \cdot x_1$, mentre un'altra come $-0.7 \cdot x_0 + 0.7 \cdot x_1$.

- **Implementazione:** Anche se il modello è deterministico, la struttura rimane quella di un MDP dove la probabilità di transizione verso lo stato calcolato è pari a 1.

Caso di Studio: Il Semaforo ("course/2000")

- **Modellazione degli Stati:** Il semaforo è rappresentato da tre stati discreti: Verde (G), Rosso (R) e Giallo (Y).
- **Ingressi (Input):**
 - ϵ : "non fare nulla", il semaforo mantiene il colore attuale.
 - σ : "cambia luce", attiva la transizione sequenziale valida (G→Y→R→G).
- **Logica Temporale:** Il cambio di stato avviene a intervalli casuali generati da una `std::uniform_int_distribution` (es. tra 60 e 120 secondi).
- **Implementazione C++:** Per gestire la rotazione dei colori in modo compatto, si possono utilizzare operatori ternari che controllano lo stato attuale e assegnano il successivo.

Introduzione alla Libreria `mocc`

- **Scopo:** Facilitare la creazione di sistemi complessi composti da molteplici unità comunicanti.
- **Componenti Timed:** I componenti possono avere un timer interno; quando il timer raggiunge lo zero, il componente riceve una notifica tramite il metodo `update()`.
- **Observer Pattern:** La libreria implementa il pattern Observer per gestire la comunicazione tra i componenti (es. il semaforo osserva un timer).

Gestione dei Timer

Esistono due modalità principali per i timer nella libreria `mocc`:

- **TimerMode::Once:** Il timer termina e deve essere resettato manualmente. È utile quando si vuole impostare una durata diversa (es. casuale) per ogni ciclo.
- **TimerMode::Repeating:** Il timer si resetta automaticamente con l'ultimo valore impostato.

Strutturazione del Progetto

- **Header `parameters.hpp`:** Viene utilizzato per definire costanti globali, tipi e variabili di simulazione (come `HORIZON` o il generatore casuale `urng`) accessibili da tutti i moduli.
- **Pragma Once:** Si preferisce `#pragma once` ai classici `include guards` per evitare definizioni multiple, pur non essendo formalmente parte dello standard C++.
- **Incapsulamento:** Lo stato dell'MDP (es. il colore della luce) dovrebbe essere leggibile ma non modificabile direttamente dal codice esterno.

Centro di Controllo e Notifiche ("course/3200")

- **Funzionamento:** Il `ControlCenter` agisce come l'entità decisionale che determina quando cambiare il colore della luce.

- **Pattern Notifier:** A differenza del semplice semaforo, il centro di controllo utilizza il pattern **Notifier** per inviare segnali ad altri componenti (come la rete) ogni volta che avviene uno switch.
- **Payload:** Viene introdotto il concetto di `Payload` (un alias di `Light`), utilizzato per garantire che solo i componenti compatibili possano ricevere e processare il messaggio inviato dal controllore.

Strong Aliases e Type-Safety

- **Definizione:** Tramite `STRONG_ALIAS`, si creano tipi distinti a partire da tipi base (es. `NetworkPayloadLight` derivato da `Light`).
- **Vantaggio:** Sebbene i valori siano intercambiabili, il compilatore li tratta come tipi diversi, permettendo connessioni tra componenti estremamente sicure e prevenendo errori logici nel cablaggio del sistema.

Modellazione della Rete ("course/3200")

- **Buffer:** La rete viene modellata come un componente dotato di un `Buffer` potenzialmente illimitato per i messaggi in entrata.
- **Ritardo (Delay):** Per simulare il comportamento reale, la rete non consegna il messaggio istantaneamente, ma attende un tempo prefissato (es. 2 secondi) prima di notificare il destinatario.
- **Limiti del Modello Base:** Inizialmente, la rete è "perfetta" e non tiene conto di congestione, perdita di pacchetti o corruzione dei dati.

Monitoraggio del Sistema

- **Il Monitor:** È un componente che osserva sia il `ControlCenter` che il `TrafficLight`.
- **Verifica Coerenza:** Il suo compito è verificare che il sistema funzioni correttamente; ad esempio, segnala un errore se il semaforo non cambia colore entro 3 secondi dall'invio del comando da parte del centro di controllo.

Introduzione dei Guasti (Faults) ("course/3300")

- **Modellazione Probabilistica dei Guasti:** Viene aggiunta una variabile di guasto utilizzando `std::bernoulli_distribution`.
- **Perdita di Messaggi:** A ogni tentativo di invio, la rete esegue un campionamento; se la distribuzione restituisce "vero" (guasto), il messaggio viene scartato e non raggiunge mai il semaforo.
- **Assenza di Ripristino:** In questa fase del modello, una volta che il messaggio è perso, il sistema non ha meccanismi per accorgersene o recuperare, rimanendo in uno stato incoerente.

Riparazione dei Guasti e Protocolli ("course/3400" - "course/3500")

- **Meccanismi di Riparazione:** Oltre alla probabilità di guasto, viene introdotta una `std::bernoulli_distribution` per la **riparazione casuale** dei messaggi.

- **Logica di Recupero:** Se un messaggio subisce un guasto, il sistema può comunque tentare di inviarlo con successo se la condizione di riparazione è soddisfatta.
- **Notifica di Errore (Fault Signal):** Nella versione più avanzata, la rete non si limita a scartare il messaggio, ma invia una notifica esplicita di `Fault` (un alias di `bool`) agli altri componenti.
- **Comportamento di Sicurezza:** Quando il `TrafficLight` riceve una notifica di `Fault`, cambia automaticamente il suo stato in `RED` (Rosso) per garantire la sicurezza del sistema in caso di guasto della rete.

Simulazione del Sistema Completo

- **Integrazione dei Componenti:** Il sistema finale mette insieme `Monitor`, `Network`, `TrafficLight` e `ControlCenter`, tutti collegati tramite il pattern Observer/Notifier.
- **Ciclo di Simulazione:** La simulazione avviene tramite un ciclo `while` che chiama ripetutamente `system.next()` finché non viene raggiunto l'orizzonte temporale (`HORIZON`) stabilito.
- **Logging:** Durante ogni passo temporale, lo stato di tutti i componenti (luce del controller, luce effettiva del semaforo, validità del monitor) viene salvato in un file di log per l'analisi successiva.

Statistiche e Valore Atteso ("course/4100")

- **Simulazione di Processo:** L'obiettivo si sposta sulla modellazione di un processo di sviluppo software suddiviso in fasi (fase 0, 1, 2).
- **Costi delle Fasi:** A ogni fase è associato un costo specifico (es. 20 unità per le fasi 1 e 3, 40 unità per la fase 2).
- **Evoluzione del Costo:** Il costo totale di una simulazione viene calcolato incrementando una variabile ogni volta che il sistema transita attraverso una determinata fase fino al completamento (fase 4).

Probabilità e Metodo Monte Carlo ("course/4200")

- **Stima della Probabilità:** Viene utilizzato il metodo Monte Carlo per calcolare la probabilità che il costo totale dello sviluppo sia inferiore a una certa soglia (es. < 100).
- **Iterazioni:** Per ottenere un risultato statisticamente significativo, la simulazione del processo viene ripetuta migliaia di volte (`ITERATIONS = 10000`).
- **Indicatori Statistici:** Per ogni set di simulazioni vengono calcolati e visualizzati tre valori fondamentali :
 1. **Media** dei costi.
 2. **Deviazione standard**.
 3. **Probabilità** (casi favorevoli / totale iterazioni).

Matrici di Transizione Casuali ("course/5100")

- **Generazione Automatica:** È possibile costruire una matrice di transizione senza definire manualmente ogni probabilità, utile per testare la robustezza di un sistema su scenari ignoti.
- **Implementazione:**
 - Si definisce un vettore di distribuzioni discrete (`vector<discrete_distribution>`), una per ogni stato .
 - Per ogni stato, si genera un vettore di "pesi" casuali tramite una distribuzione uniforme reale .
 - Questi pesi vengono passati alla `discrete_distribution` , che normalizza automaticamente i valori affinché la somma delle probabilità sia 1 .
- **Stati Numerati:** Gli stati sono identificati da indici interi (da 0 a `STATES_SIZE - 1`), facilitando l'accesso diretto alla riga corrispondente della matrice.

Modello di Sviluppo e Rilevamento Errori ("course/5200")

- **Struttura della Fase:** Ogni fase del processo di sviluppo ha una durata fissa (es. 4 giorni) .
- **Esiti della Fase (Outcomes):** Al termine di ogni fase, un "oracolo" determina la situazione reale, che viene classificata in tre tipi :
 1. **NO_ERROR:** Nessun errore introdotto.
 2. **NO_ERROR_DETECTED:** Errore introdotto ma non trovato dal team di testing.
 3. **ERROR_DETECTED:** Errore introdotto e correttamente identificato.
- **Logica di Avanzamento:**
 - Se non ci sono errori o non vengono rilevati, il team procede alla fase successiva .
 - Se viene rilevato un errore, il team deve ripetere la fase attuale (con alta probabilità, es. 0.8) o tornare a una delle fasi precedenti (con probabilità uniforme) per correggere il problema .

Parametri da File Esterni

- **Configurazione:** Invece di cablare le probabilità nel codice, il simulatore legge i parametri (probabilità di introdurre un errore e probabilità di non rilevarlo) da un file CSV (`probabilities.csv`) .
- **Flessibilità:** Questo approccio permette di simulare diversi scenari (es. team esperti vs. junior) semplicemente cambiando i dati nel file .

Struttura del Codice di Simulazione

- **Monitoraggio del Progresso:** Viene utilizzato un vettore `progress` per contare i giorni trascorsi nella fase attuale.
- **Uso di Assert:** Viene impiegata la funzione `assert` per verificare che il numero di distribuzioni caricate dal file corrisponda effettivamente al numero di fasi previste dal modello .
- **Logging Avanzato:** Il file di log (`log.csv`) registra per ogni istante temporale la fase attuale, il progresso e l'esito dei controlli di qualità .

Ottimizzazione e KPI ("course/5300" - "course/5400")

- **Simulazione per il Decision Making:** È possibile manipolare i parametri del modello per riflettere scenari reali; ad esempio, un team più esperto ha una probabilità inferiore di introdurre errori ma un costo orario più elevato.
- **Ricerca del Set Ottimale:** Ripetendo la simulazione con diversi parametri casuali, si possono identificare le configurazioni che minimizzano i tempi di sviluppo (approccio simile al *simulated annealing*).
- **Key Performance Index (KPI):** Si possono assegnare costi specifici a ogni parametro per analizzare il trade-off tra costi e tempi, ottimizzando il processo in base agli obiettivi aziendali.

Esempi di Sistemi Complessi ("course/6100" - "course/6300")

Il documento menziona tre casi studio avanzati che integrano tutti i concetti precedenti:

- **Pompa d'insulina:** Esempio di sistema critico dove la precisione del modello è vitale.
- **Buffer:** Gestione delle code e dei flussi di dati.
- **Server:** Modellazione del carico e delle risposte alle richieste dei client.

Guida alla Prova d'Esame (Sezione 4)

L'autore riassume lo svolgimento di ogni sistema software in 4 passaggi fondamentali :

1. **Lettura dei parametri:** Acquisizione dei dati da file esterni (solitamente `.txt` o `.csv`).
2. **Inizializzazione:** Istanziatura degli MDP e creazione dei collegamenti (connessioni) tra i componenti .
3. **Simulazione:** Esecuzione del ciclo principale della simulazione.
4. **Salvataggio:** Scrittura dei risultati e dei log su file di output.

Consigli Tecnici per l'Esame

- **Lettura da File:** Viene suggerito l'uso di `std::ifstream` che, combinato con uno `switch`, permette di leggere variabili di tipo diverso (`char`, `float`, `int`) in base a etichette nel file (es. `'A'`, `'B'`, `'N'`) .
- **Gestione dei Parametri:** È prassi dichiarare i parametri come `static` in un file `parameters.hpp` per renderli accessibili globalmente a tutte le classi senza doverli passare nei costruttori .

- **Loop di Gioco:** La simulazione deve essere gestita come un "game loop", dove un oggetto `Stopwatch` misura il tempo trascorso e interrompe l'esecuzione al raggiungimento del `HORIZON` .

Esempio Pratico: Classe Employee ("practice/1")

- **Struttura:** Un dipendente è modellato come un osservatore del tempo (`Observer<T>`) e un notificatore del completamento dei task (`Notifier<TaskDone>`) .
- **Matrice Dinamica:** La matrice di transizione del dipendente viene costruita matematicamente nel costruttore utilizzando parametri come `A, B, C, D` per calcolare le probabilità di avanzamento tra le fasi .
- **Logica di Stato:** Quando il dipendente raggiunge l'ultima fase (N-1), salva le statistiche di completamento e notifica il sistema del compito svolto e del costo associato .